

NLP - Mini Projet - Classification de texte

Rapport

Orlan MONJOLY, Romain BOWE
orlan.monjoly@etu.cyu.fr, romain.bowe@etu.cyu.fr

1 Choix du thème et du dataset

Pour ce projet, nous avons décidé de nous orienter vers le domaine culinaire. Après plusieurs recherches, nous avons choisi de faire un classificateur de texte qui, à partir d'une recette, détermine l'origine de la cuisine. Nous avons ensuite trouvé plusieurs datasets intéressants à analyser, mais tous avec différents défauts, le principal étant le manque de données. Nous avons donc décidé d'utiliser celui qui contient le plus de données possible : le dataset « What's cooking » (lien : <https://www.kaggle.com/competitions/whats-cooking/overview>), qui a été utilisé pour un tournoi de prédiction sur Kaggle il y a 11 ans.

Ce dataset nous fournit un fichier json train.json contenant 39774 listes d'ingrédients et 20 classes différentes. De plus, vu l'origine du dataset, nous avons décidé de faire confiance à sa qualité en admettant qu'il n'y a pas de doublons.

1.1 Analyse du dataset

La première chose que nous regardons est la répartition des 20 classes dans le dataset :

```
italian : 7838 (19.71%)
mexican : 6438 (16.19%)
southern_us : 4320 (10.86%)
indian : 3003 (7.55%)
....
irish : 667 (1.68%)
jamaican : 526 (1.32%)
russian : 489 (1.23%)
brazilian : 467 (1.17%)
```

Les résultats ci-dessus montrent un déséquilibre certain entre les différentes classes du dataset : en effet, 2 classes sur 20 (italian et mexican) représentent plus de 35% de celui-ci. Lors des différents apprentissages, nous pouvons donc déjà dire qu'il sera plus dur de trouver les classes les moins présentes, telles que brazilian et russian, à cause du peu de données que l'on a sur celles-ci.

Voici la forme d'une donnée dans le fichier json :

```
{
  "id": 17636,
  "cuisine": "italian",
  "ingredients": [
    "tomato sauce",
    "shredded carrots",
    "spinach",
    "part-skim mozzarella cheese",
    "italian seasoning",
    "english muffins, split and
    toasted",
    "chopped onion",
    "vegetable oil cooking spray",
    "chopped green bell pepper"
  ]
},
```

2 Système sans machine learning

2.1 Vocabulaire

Pour le système sans machine learning, nous avons décidé de partir d'un IDF afin de définir un vocabulaire contenant les 30 ingrédients les plus présents dans chaque classe, ce qui nous donne une base de vocabulaire assez grande pour analyser chaque recette. Exemple :

```
'greek': ['salt', 'olive oil', 'feta
cheese crumbles', 'dried oregano',
'garlic cloves', 'ground black pepper',
'extra-virgin olive oil', 'pepper',
'garlic', 'fresh lemon juice', 'feta
cheese', 'cucumber', 'lemon juice',
'purple onion', 'onions', 'tomatoes',
'water', 'lemon', 'fresh parsley', 'fresh
dill', 'red wine vinegar', 'butter',
'all-purpose flour', 'black pepper',
'kosher salt', 'ground cinnamon',
'kalamata', 'eggs', 'minced garlic',
'greek yogurt']
```

En prenant l'union des 30 ingrédients les plus présents dans chacune des 20 classes, nous obtenons :

nous notre vocabulaire final, qui contient 179 mots.

2.2 Définition des vecteurs de chaque classe

À partir de ce vocabulaire, nous pouvons définir un vecteur représentatif de chaque classe contenant 179 valeurs, une pour chaque mot. La valeur est égale à 0 si l'ingrédient n'est pas dans les 30 ingrédients les plus présents de la classe, et sinon égale à $\ln\left(\frac{\text{nb classe}}{\text{nb classe avec ingrédient}}\right)$. Cela s'interprète ainsi : si un ingrédient est présent dans le top de beaucoup de classes différentes, il affectera moins la décision pour les classes qui le contiennent. Exemple (début du vecteur pour la cuisine grecque) :

```
'greek': array([0., 0., 0., 0.7985077 ,
0.,0., 0., 0., 0., 0.,0., 0., 0., 0.,
0.,0., 0.597837, 0., 0.51082562, 0.,0.,
0., 0., 0., 0.,0., 0., 0., 0., 0.0., 0.,
0., 0., 0.0., 0., 0., 0., 0.
.....
```

2.3 Calcul de la classe d'une recette

Pour classifier une recette, nous prenons chaque ingrédient qu'elle contient et, s'il est dans le vocabulaire, nous mettons à 1 la valeur correspondante du vecteur de la recette. Nous obtenons ainsi un vecteur rempli de 0 et de 1. Pour chaque classe, nous appliquons ensuite une combinaison linéaire des deux vecteurs pour obtenir une valeur intermédiaire, puis nous appliquons un softmax sur toutes ces valeurs pour obtenir le résultat final. On peut voir cela comme un produit matriciel qui donne un vecteur sur lequel on applique la fonction softmax.

$$\underbrace{\begin{pmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,179} \\ w_{2,1} & w_{2,2} & \dots & w_{2,179} \\ \vdots & \vdots & \ddots & \vdots \\ w_{20,1} & w_{20,2} & \dots & w_{20,179} \end{pmatrix}}_{W \in \mathbb{R}^{20 \times 179}} \cdot \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_{179} \end{pmatrix}}_{x \in \{0,1\}^{179}} = \underbrace{\begin{pmatrix} s_1 \\ s_2 \\ \vdots \\ s_{20} \end{pmatrix}}_{s \in \mathbb{R}^{20}}$$

$$s_k = \sum_{j=1}^{179} w_{k,j} x_j \quad p_k = \frac{e^{s_k}}{\sum_{i=1}^{20} e^{s_i}}$$

2.4 Résultats

Voici le tableau des différentes métriques :

	Ac- cura- cy	Préc. ma- cro	Rap- pel ma- cro	F1 ma- cro	F1 pond.
Train	0.4800	0.4172	0.4516	0.4104	0.5027
Val	0.4804	0.4192	0.4485	0.4099	0.5022
Test	0.4739	0.4065	0.4314	0.3961	0.4971

Tableau 1. – Résultats du modèle sans apprentissage.

Dans les résultats ci-dessus, les moyennes macro sont les moyennes des résultats de chacune des 20 classes, et le F1 pond. prend en compte le poids de chaque classe, car celles-ci ne sont vraiment pas bien réparties dans le dataset.

Voir en annexe pour la matrice de confusion.

Résultats de la précision par classe :

Classe	Préc	Rappel	F1	Support
mexican	0.8735	0.6856	0.7683	967
indian	0.7051	0.7051	0.7051	451
chinese	0.7000	0.5920	0.6415	402
.....				
viet	0.1894	0.2000	0.1946	125
irish	0.1504	0.1980	0.1709	101

Ce que nous pouvons observer dans ces résultats, c'est que la précision monte jusqu'à 87% pour certaines cuisines (mexican) et descend jusqu'à 15% pour d'autres (irish). Cela donne une moyenne autour de 50% de précision.

3 Modèle de deep learning

Pour le modèle de deep learning, nous repartons du vocabulaire construit avec l'IDF dans la partie sans apprentissage. Cette fois, au lieu de créer des vecteurs pour chaque classe, nous construisons un réseau de neurones qui prend en entrée le vecteur de vocabulaire de notre recette et qui a en sortie 20 neurones, un pour chaque classe, sur lesquels nous appliquons un softmax pour obtenir une seule classe. Entre les deux, nous avons utilisé 2 couches cachées, dont la première est composée de 128 neurones et la seconde de 64. (Le choix de 2 couches cachées est totalement arbitraire et le nombre de neurones par couche est une proposition de ChatGPT.)

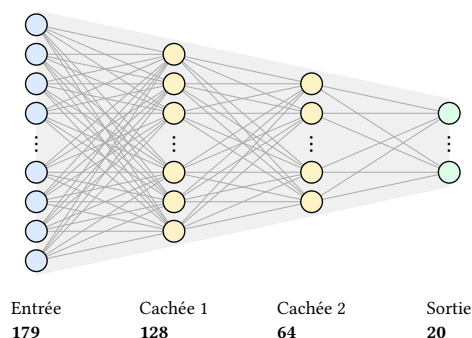


Fig. 1. – Architecture du réseau : 179 entrées, deux couches cachées (128 et 64 neurones) et 20 sorties. Le nombre de neurones dessinés est réduit.

Paramètres utilisés :

- loss : CrossEntropyLoss
- optimizer : Adam avec $\text{lr}=1\text{e-}3$
- nombre d'epochs : 30

3.1 Résultats

Voici le tableau des différentes métriques :

	Ac- cura- cy	Préc. ma- cro	Rap- pel ma- cro	F1 ma- cro	F1 pond.
Train	0.6871	0.7047	0.5656	0.6084	0.6766
Val	0.6587	0.6429	0.5179	0.5540	0.6440
Test	0.6526	0.6352	0.5082	0.5460	0.6388

Tableau 2. – Résultats du modèle avec apprentissage (réseau de neurones).

Voir en annexe pour la matrice de confusion.

Résultats de la précision par classe :

Classe	Préc	Rappel	F1	Support
mexican	0.8143	0.8480	0.8308	967
indian	0.7932	0.8337	0.8130	451
...				
spanish	0.4583	0.1477	0.2234	149
british	0.5172	0.1230	0.1987	122

Ces résultats nous montrent que la méthode par apprentissage est bien meilleure que celle sans apprentissage, et que l'amélioration porte surtout sur la précision des pires classes, qui, au lieu d'être autour de 15%, sont maintenant à 50%.

3.1.1 Analyse de 5 erreurs

Toutes les erreurs que nous avons trouvées sur le modèle sont la conséquence de la différence de proportion des types de cuisine dans les données : le modèle est plus entraîné à trouver correctement une cuisine italienne ou mexicaine, qui sont majoritaires dans le dataset.

Erreur 1

Recette 1087

Vraie cuisine : indian (proba donnée par le modèle : 0.038)

Cuisine prédite : mexican (proba : 0.892)

Ingrédients : tomatoes, salt, chili powder, canola oil, garlic, tamarind extract, onions

Ici, on voit très clairement le problème de disproportion des classes dans le dataset, avec la cuisine mexicaine à 16% tandis que la cuisine indienne est à 7.5%.

Erreur 2

Recette 4489

Vraie cuisine : thai (proba donnée par le modèle : 0.057)

Cuisine prédite : mexican (proba : 0.886)

Ingrédients : lime juice, Thai fish sauce, brown sugar, grapefruit, avocado, purple onion, chopped cilantro fresh, red chili peppers, uncook medium shrimp, peel and devein

Pour cette erreur, on voit très clairement le problème de la majorité de la cuisine mexicaine dans le dataset. D'autre part, en augmentant la taille du vocabulaire, « Thai fish sauce » aurait pu y apparaître et jouer alors un rôle décisif dans le choix.

Erreur 3

Recette 4526

Vraie cuisine : vietnamese (proba donnée par le modèle : 0.126)

Cuisine prédite : thai (proba : 0.871)

Ingrédients : groundnut, lime juice, spring onions, rice flour, ground turmeric, fish sauce, fresh coriander, vegetables, garlic cloves, coconut milk, caster sugar, shiitake, sea salt, beansprouts, red chili peppers, water, lettuce leaves, king prawns, onions

Dans cette erreur, il ne s'agit pas d'un problème de proportion dans le dataset, mais de similitude entre les cuisines.

Erreur 4

Recette 5752

Vraie cuisine : korean (proba donnée par le modèle : 0.046)

Cuisine prédite : chinese (proba : 0.884)

Ingrédients : water, sesame oil, sugar, light soy sauce, beef tenderloin, dark soy sauce, minced ginger, crushed garlic, black pepper, green onions, white sesame seeds

Malgré la différence de proportion entre la cuisine chinoise et la cuisine coréenne, ces deux cuisines sont similaires car elles viennent de la même région, et l'on peut en déduire que le modèle trouve la bonne région.

Erreur 5

Recette 5896

Vraie cuisine : russian (proba donnée par le modèle : 0.001)

Cuisine prédite : southern_us (proba : 0.912)

Ingrédients : sugar, salt, baking powder, oil, large eggs, all-purpose flour, buttermilk

Ici, les ingrédients sont présents dans toutes les cuisines, donc le modèle a choisi, par probabilité, une des classes les plus présentes.

4 Transformer de classification

4.1 Présentation du modèle choisi

Nous avons choisi le transformer RoBERTa (base), créé par Facebook en 2019. Ce nom est l'acronyme de Robustly Optimized BERT Approach, et le modèle est le successeur de BERT (Bidirectional Encoder Representations from Transformers), sorti par Google en 2018. Il a été entraîné sur 160 Go de texte en anglais, comporte environ 125M de paramètres et utilise un Byte Pair Encoding au niveau des octets (byte-level BPE) pour la tokenisation.

4.2 Tokenisation et longueur maximale

Le tokenizer de RoBERTa est un BPE byte-level. Pour tokeniser, nous commençons par charger le modèle RoBERTa et son tokenizer associé, puis

nous calculons la longueur maximale du texte d'après la distribution réelle. Pour cela, nous convertissons les textes en une liste, puis nous calculons le nombre de tokens par texte et le stockons dans un tableau numpy. Nous calculons ensuite les percentiles, c'est-à-dire le nombre maximal de tokens utilisé par p% des textes, et nous plafonnons à 128. Nous trouvons que, pour p% = 99.9, 110 tokens au maximum sont utilisés ; nous retenons donc cette valeur comme longueur maximale, ce qui revient à tronquer 0.0987% des textes.

4.3 Paramètres d'entraînement

Paramètres	Phase 1 : tête seule	Phase 2 : fine-tuning complet
Paramètres entraînés	tête unique- ment (en- deur gelé)	tous
Epochs	4	4
Learning rate	1e-3	2e-5
Batch size (train)	64	32
Batch size (éval)	128	128
Weight decay	0.01	0.01
Warmup	6% des pas	6% des pas
Scheduler	linéaire (dé- croissance)	linéaire (dé- croissance)

Tableau 3. – Paramètres d'entraînement

4.4 Résultats

Modèle	Accuracy	Macro-F1
Phase 1 : tête seule	0.567288	0.247859
Phase 2 : fine-tuning	0.776966	0.671193

Tableau 4. – Résultats des deux phases d'entraînement.

Voir en annexe pour la matrice de confusion.

5 Conclusion

Pour conclure, parmi les 3 modèles différents, nous choisirions le modèle avec apprentissage, car d'une part il est meilleur que celui sans apprentissage sans demander plus de temps d'entraînement (environ 30 secondes). D'autre

part, le problème du transformer dans cette situation est qu'il prend beaucoup plus de place et de temps à entraîner pour un résultat très similaire à celui du modèle avec apprentissage.

6 Annexe

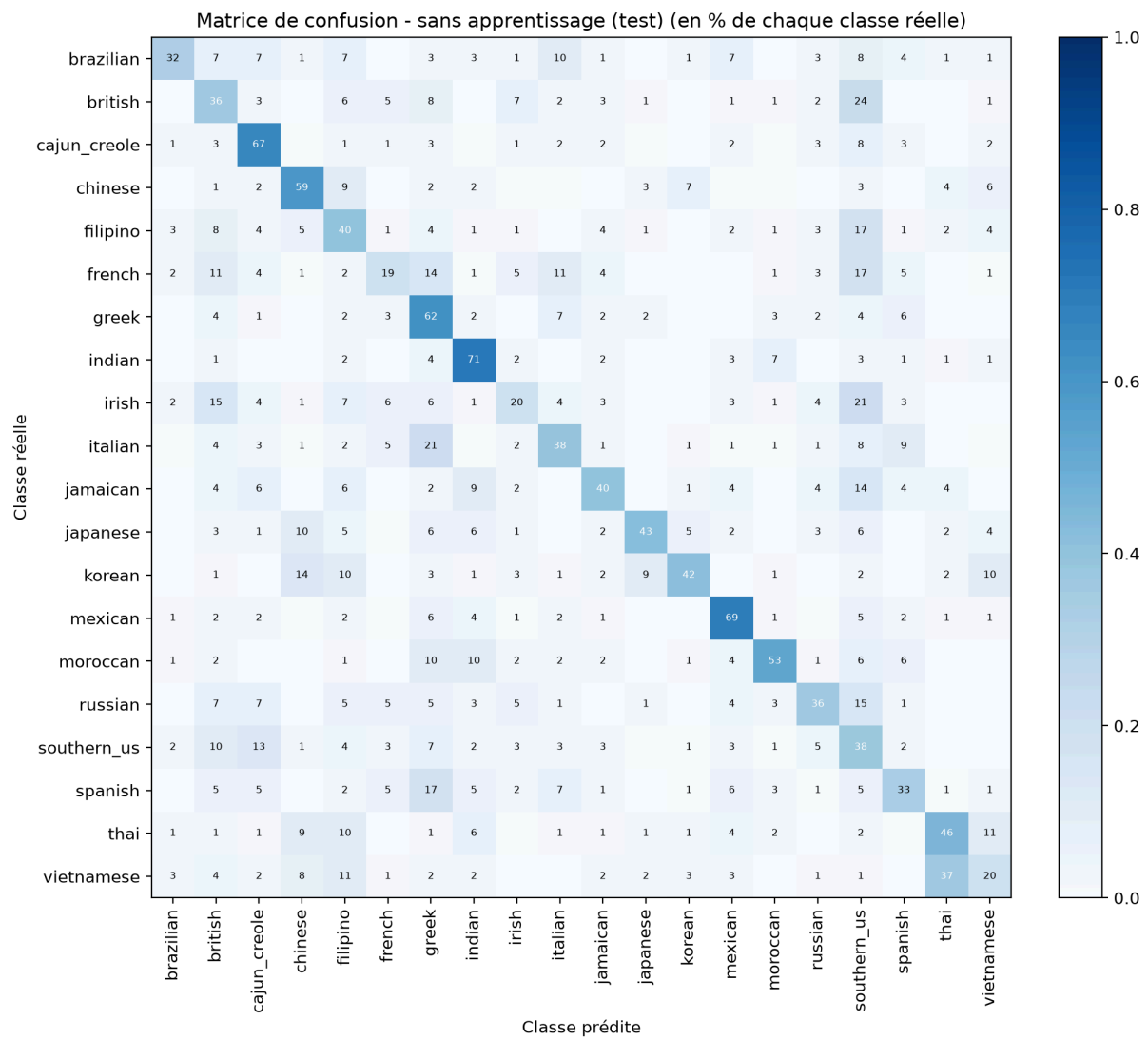


Fig. 2. – Matrice de confusion du modèle sans apprentissage (jeu de test).

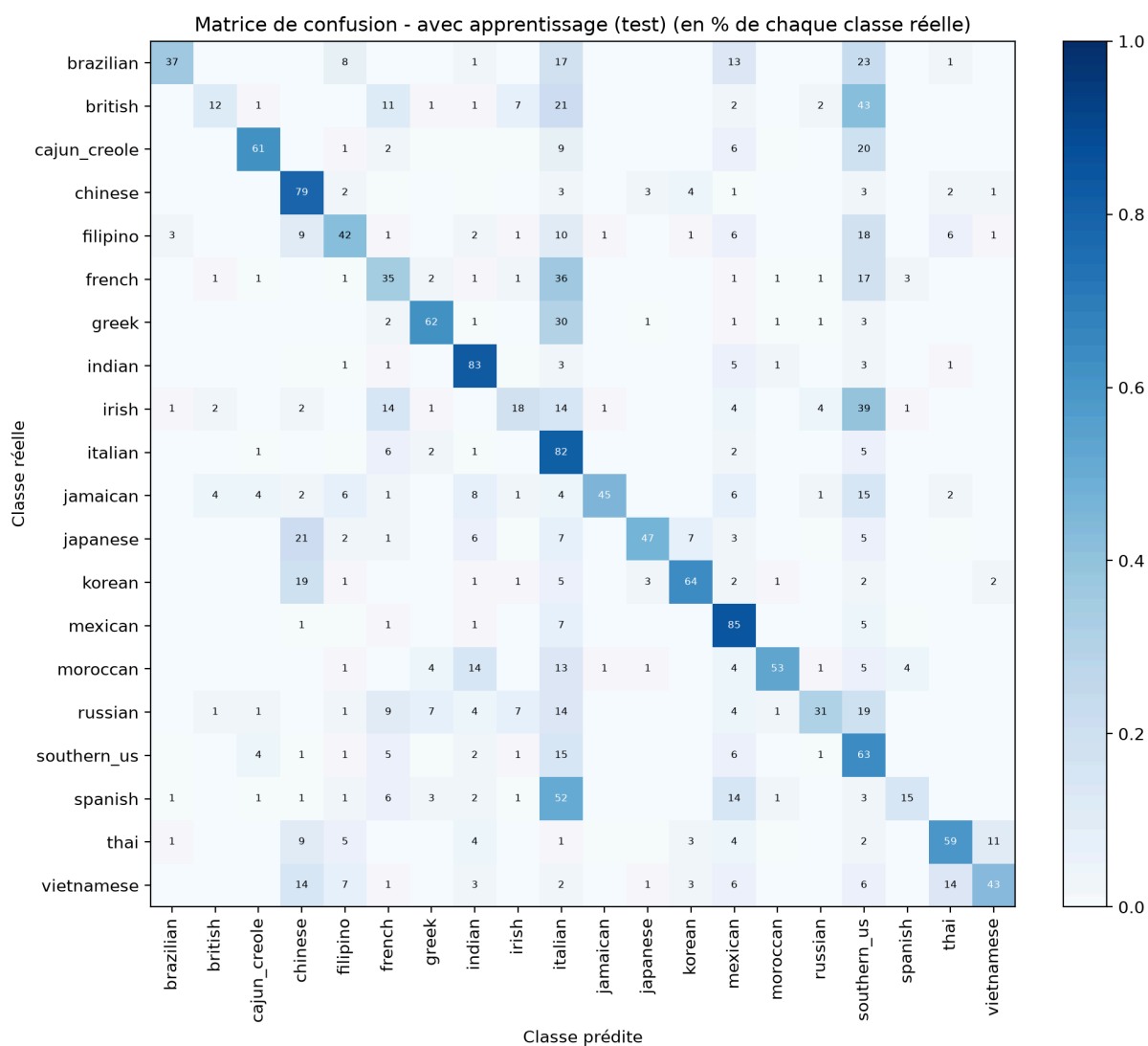


Fig. 3. – Matrice de confusion du réseau de neurones (jeu de test).

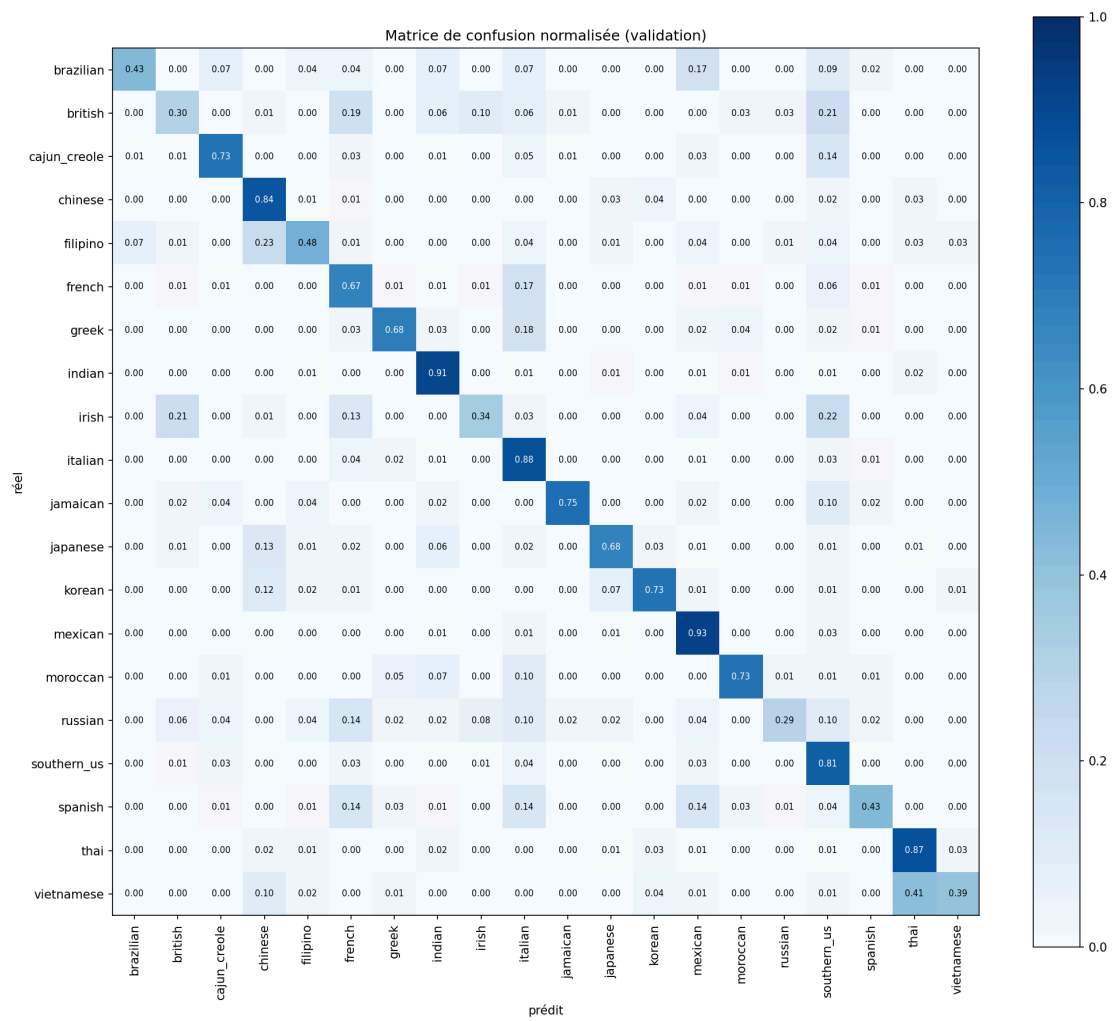


Fig. 4. – Matrice de confusion de la phase d'entraînement de la tête avec RoBERTa.